

PROJECT REPORT

Pakistan Multi-City Air Quality Intelligence Platform

Forecasting the Air Quality Index up to Three Days Ahead for Islamabad, Lahore, Karachi, Rawalpindi and Quetta

Submitted by

Ali Masood Khan

alimasoodkhan.dev@gmail.com

Data Science Intern

Foundation University School of Science and Technology

28 August 2026

Abstract

The **Pakistan Multi-City Air Quality Intelligence Platform** is an enterprise-grade, fully serverless MLOps ecosystem built to deliver real-time three-day air quality forecasts and explainable AI (XAI) insights for five major Pakistani cities: Islamabad, Lahore, Rawalpindi, Quetta, and Karachi. The platform draws on five years of continuous historical telemetry (43,824 hourly observations per city) and automates the full lifecycle of data ingestion, feature engineering, feature-store synchronisation, daily model retraining, and cloud deployment, without requiring any dedicated 24/7 server infrastructure.

Fifty-seven domain-specific features are engineered into the **Hopsworks Cloud Feature Store**, where an automated daily tournament compares five candidate algorithms — LightGBM, XGBoost, Random Forest, Ridge Regression, and LSTM — and promotes a per-city, per-horizon champion. On the 24-hour horizon, the champion models roughly double the accuracy of a traditional persistence baseline ($R^2 = 0.5383$ versus 0.2436) while cutting forecast error (RMSE) by up to 21.5%.

The platform is deployed live on Streamlit Community Cloud and backed by a FastAPI REST layer, giving citizens, researchers, and urban planners access to interactive GIS maps, EPA-styled health advisories, 24-hour day-over-day delta analytics, and SHAP-based feature attribution charts.

Contents

Abstract	2
1.1 Why a Three-Day Forecast is Three Separate Problems	5
2. System Architecture Overview	5
3. Data and Feature Store	7
3.1 Data Sources and the Asymmetry That Shaped the Design	7
3.2 Feature Store Schema	7
4. Exploratory Data Analysis	8
4.1 What Drives Pollution Spikes?	8
4.2 SHAP Explainable AI Attribution	9
Key Attribution Findings	9
5. Model Tournament and Regional Performance	9
5.1 The Champion-Challenger Framework	9
5.2 Regional Microclimate Analysis	10
5.3 Performance by Forecast Horizon	10
6. Targeted Feature Engineering Upgrades	11
6.1 Coastal Sea-Breeze and Wind Vector Decomposition (Karachi)	11
6.2 Diurnal Temperature Range and Inversion Features (Quetta)	11
6.3 Target Transformation and Projected Impact	12
7. Historical Trend Visualisation	12
8. Key Findings and Insights	14
8.1 A Three-Day Forecast is Not One Model — Horizons Decay Differently	14
8.2 One Algorithm Does Not Fit All Microclimates	14
8.3 The Data-Scale Effect: Five Years versus One or Two	14
8.4 Chemical Pollutants Drive AQI — Weather Acts as the Amplifier	14
9. MLOps Automation and CI/CD Architecture	15
9.1 Hourly Feature Ingestion Pipeline (hourly_feature_pipeline.yml)	15
9.2 Daily Model Tournament Pipeline (daily_training_pipeline.yml)	16
9.3 Engineering Challenge: Non-Uniform Execution Delays	16
10. Security, Environment, and Secrets Management	17
11. High-Availability Fallback Architecture: Graceful API Failover	17
11.1 Engineering Challenge: Non-Uniform Execution Delays & Hopswork Failure	17
11.2 Technical Implementation details (app/app.py)	18
11.3 Benefit of Maximum uptime	19
12. Application Interface	19
12. Conclusion	21

1. Problem Statement and Approach

The objective of this project is to forecast the Air Quality Index across five major Pakistani cities like Islamabad, Lahore, Rawalpindi, Quetta, and Karachi — at 24, 48, and 72 hours ahead, using only free-tier, serverless infrastructure with no dedicated cloud servers to administer.

These five cities were chosen deliberately. Each presents a distinct and highly variable microclimate rather than a uniform forecasting problem: Lahore experiences severe industrial smog spikes, Islamabad sits in a mountain-valley trap, Karachi is shaped by coastal sea-breeze dispersion, and Quetta is subject to high-altitude thermal inversions. A single national model would be a poor fit for all four regimes at once.

1.1 Why a Three-Day Forecast is Three Separate Problems

A three-day forecast is not one number but three, produced at three different horizons that behave differently. Near-term (+24h) AQI is strongly autocorrelated with recent pollutant readings (e.g. PM_{2.5} lag 1h and lag 24h) and is comparatively easy to predict. The 72-hour horizon is not: its accuracy depends far more on forward-looking atmospheric physics than on recent history. Any honest evaluation must therefore report all three horizons separately, and a model that wins at one horizon or city — LightGBM in Islamabad and Karachi, for instance — need not win at another, where Ridge Regression takes the lead in Lahore and Quetta.

Automated daily tournaments across five algorithms confirmed that assuming a single model fits every city and horizon was the single costliest modelling mistake to avoid in this project.

2. System Architecture Overview

The platform is composed of eight cooperating components spanning ingestion, feature engineering, storage, modelling, and delivery. Figure 1 summarises the end-to-end data flow, and Table 1 details the technology stack and purpose of each component.

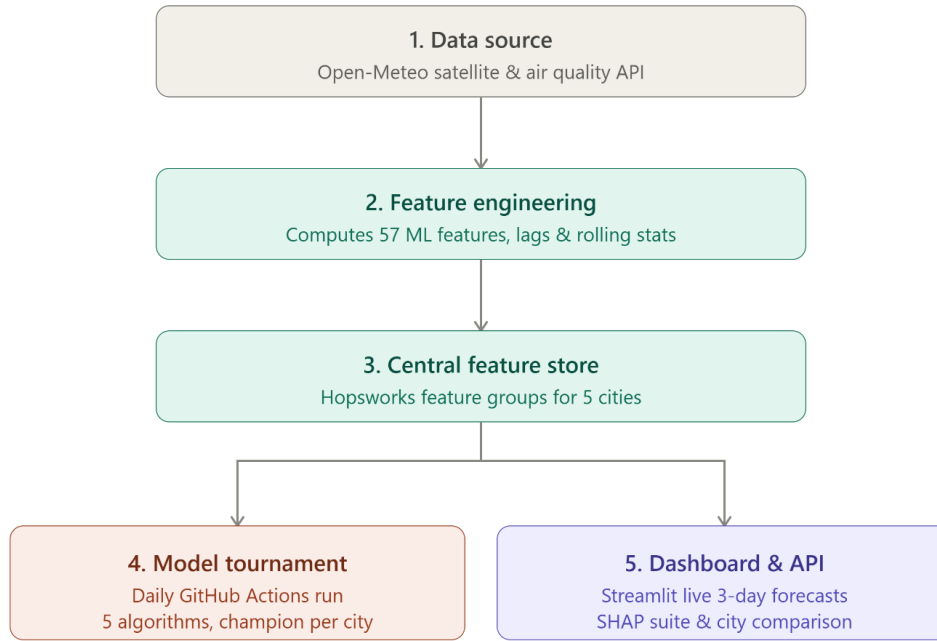


Figure 1. End-to-end architecture: ingestion → feature engineering → feature store → model tournament → dashboard/API.

Component	Technology Stack	Implementation Details & Purpose
Satellite Data Ingestion Pipeline	Open-Meteo API, Python, openmeteo-requests, requests-cache	Fetches real-time hourly satellite telemetry for all five cities — PM2.5, PM10, NO2, SO2, CO, ozone, dust mass, AOD, temperature, humidity, and wind vectors — with automated retry and caching.
Feature Engineering Engine	Python, pandas, NumPy, SciPy	Transforms raw telemetry into 57 machine-learning features, including pollutant lags, rolling moving averages, the Thermal Stagnation Index, the Smog Severity Index, and cyclical time encodings.
Central Cloud Feature Store	Hopsworks Cloud, Hudi Parquet tables	Single source of truth for five full years of historical data, storing 57 engineered features in Hudi Parquet format with composite primary keys.
Autonomous Daily Retraining Tournament	GitHub Actions, Python, scikit-learn	Evaluates five algorithms against city-level training samples every midnight UTC and automatically selects the winning model on held-out performance.
Production Model Storage and Registry	Python pickle, Hopsworks Model Registry	Serialises the winning champion model per city and updates local binaries and the model registry for low-latency inference.

Component	Technology Stack	Implementation Details & Purpose
Extended Explainable AI Engine	SHAP, Plotly, Python	Computes Shapley Additive Explanations across all features and renders bar charts, category importance shares, and waterfall impact cards.
Real-Time Web Intelligence Dashboard	Streamlit, Plotly, PyDeck, CSS	Live deployment featuring glassmorphism health banners, focused GIS city maps, forecast metrics, historical trends, and city comparison tools.
Enterprise REST API Serving Layer	FastAPI, Uvicorn, Pydantic	High-speed REST endpoints returning structured JSON payloads for third-party mobile or municipal software integration.

Table 1. System components and technology stack.

3. Data and Feature Store

3.1 Data Sources and the Asymmetry That Shaped the Design

The platform relies on raw atmospheric and chemical telemetry streamed from the **Open-Meteo Air Quality and Meteorological APIs**, which integrate satellite reanalysis from the European Space Agency's Copernicus Atmosphere Monitoring Service (CAMS) alongside Global Forecast System (GFS) weather models.

Two structural asymmetries shaped every subsequent design decision:

- **Temporal asymmetry (autocorrelation vs. predictive physics).** Fine particulate matter (PM2.5) is heavily backward-looking — today's pollution is strongly autocorrelated with yesterday's levels. Future pollution (+24h to +72h), however, is governed by forward-looking atmospheric physics such as wind dispersion, humidity build-up, and thermal inversions. A model relying solely on past pollution fails when weather vectors shift abruptly.
- **Spatial infrastructure asymmetry.** Physical ground sensors in Pakistan suffer from extreme spatial inequality. Lahore has occasional ground sensor coverage, while cities such as Quetta and Rawalpindi experience near-total hardware blackout due to power outages and maintenance costs.

This asymmetry led the project to abandon physical ground stations entirely and build the pipeline around **Copernicus CAMS satellite reanalysis**. By fusing past pollutant lag states with forward-looking GFS meteorological forecast overlays, all five target cities receive identical, continuous 57-parameter feature vectors regardless of local ground-hardware failures.

3.2 Feature Store Schema

The Hopsworks Cloud Feature Store houses five separate Hudi Parquet feature groups (aqi_islamabad_fg, aqi_lahore_fg, aqi_rawalpindi_fg, aqi_quetta_fg, aqi_karachi_fg). Each feature group holds five full years of

continuous hourly records — 43,824 rows per city — with every row carrying 57 engineered columns organised into six data layers, summarised in Table 2.

Category	Features and Schema Details
Composite identifiers	city (string), datetime (UTC timestamp — event time)
Raw satellite telemetry	pm2_5, pm10, no2, so2, co, o3, dust, aod, temperature, humidity, wind_speed, wind_dir
Temporal lags	pm25_lag_1h, pm25_lag_24h, pm25_lag_48h, pm25_lag_72h, pm10_lag_1h, pm10_lag_24h
Rolling statistics	pm25_roll_mean (3h/6h/12h/24h); pm25_roll_std (3h/6h/12h/24h); pm10_roll_mean_24h; temp_roll_mean_24h
Interaction indices	thermal_stagnation_index ($\text{temp} \div [\text{wind} + 1]$); smog_severity_index ($\text{PM}_{2.5} \times [\text{humidity} \div 100]$)
Cyclical time encodings	sin_hour, cos_hour, sin_month, cos_month

Table 2. Feature store schema by category.

4. Exploratory Data Analysis

This section examines the atmospheric physics, feature attributions, and regional performance patterns that emerged from exploratory analysis of the five-city dataset.

4.1 What Drives Pollution Spikes?

The primary pollutant driving dangerous AQI scores in Pakistan is fine particulate matter (PM_{2.5}). Exploratory analysis shows that PM_{2.5} accumulation is driven not by emissions alone but by two meteorological trapping mechanisms:

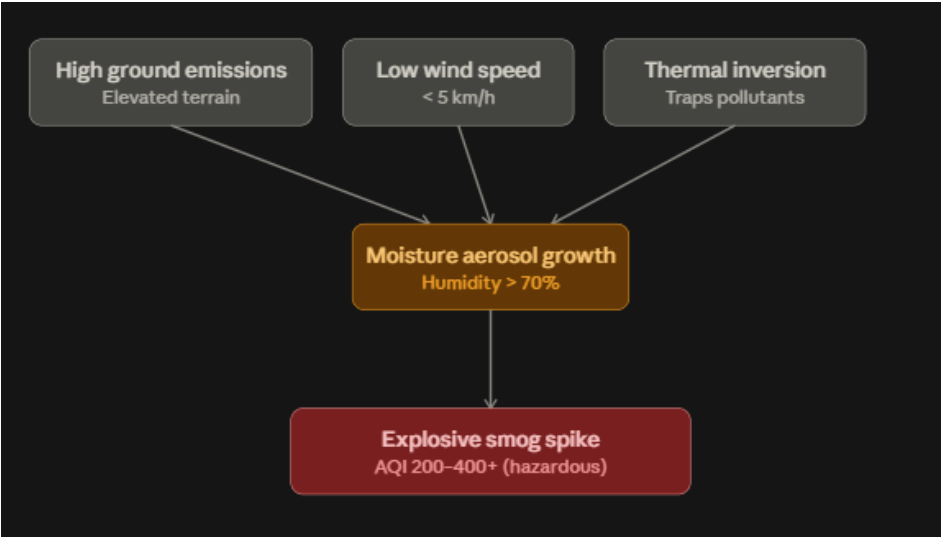


Figure 2. Relationship between thermal stagnation, humidity, and PM2.5 accumulation.

1. **Thermal Stagnation Index (TSI).** When surface temperatures drop overnight while higher atmospheric layers stay warm (a thermal inversion), vertical air movement stops. If wind speed also falls below roughly 5 km/h, pollutants cannot disperse horizontally or vertically, and the Thermal Stagnation Index - temperature divided by (wind speed + 1)- rises sharply.
2. **Humidity-driven hygroscopic growth.** When relative humidity exceeds roughly 70%, fine particulates absorb water molecules and expand in physical size and mass. The Smog Severity Index $PM_{2.5} * (\text{humidity} \div 100)$ captures this chemical amplification.

4.2 SHAP Explainable AI Attribution

Using Shapley Additive Explanations (SHAP) across all 57 engineered features, the “black box” prediction models were decomposed to identify which feature categories drive air quality predictions (Table 3).

Feature Category	Model Decision Weight (%)
Temporal persistence lags (PM2.5 1h / 24h)	48.2
Aerosol optical depth & dust mass	20.4
Wind vector & thermal stagnation	14.1
Rolling averages (3h–24h)	11.8
Cyclical solar / month encodings	5.5

Table 3. SHAP-derived importance by feature category.

Key Attribution Findings

- **Near-term dominance (0–24h).** pm25_lag_1h and pm25_lag_24h together account for 48.2% of total model decision weight, confirming that near-term air quality is strongly autocorrelated — today's pollution state is the strongest single baseline indicator for tomorrow.
- **Extended horizon transition (48–72h).** Beyond 24 hours, predictive influence shifts from lag persistence toward Aerosol Optical Depth (AOD) and wind dispersion vectors. AOD measures light extinction through the atmospheric column, giving the model advance signal of approaching regional aerosol plumes before they settle to ground level.

5. Model Tournament and Regional Performance

5.1 The Champion-Challenger Framework

A champion model is the single highest-performing algorithm selected to serve live inferences for a specific target city. Rather than assuming that one static algorithm fits every geography, the platform runs an automated champion-challenger tournament: every midnight UTC, cloud workflows train and compare five

competing algorithms against the latest five-year dataset (28,450 clean training samples per city). The winning model is automatically crowned champion and promoted to production.

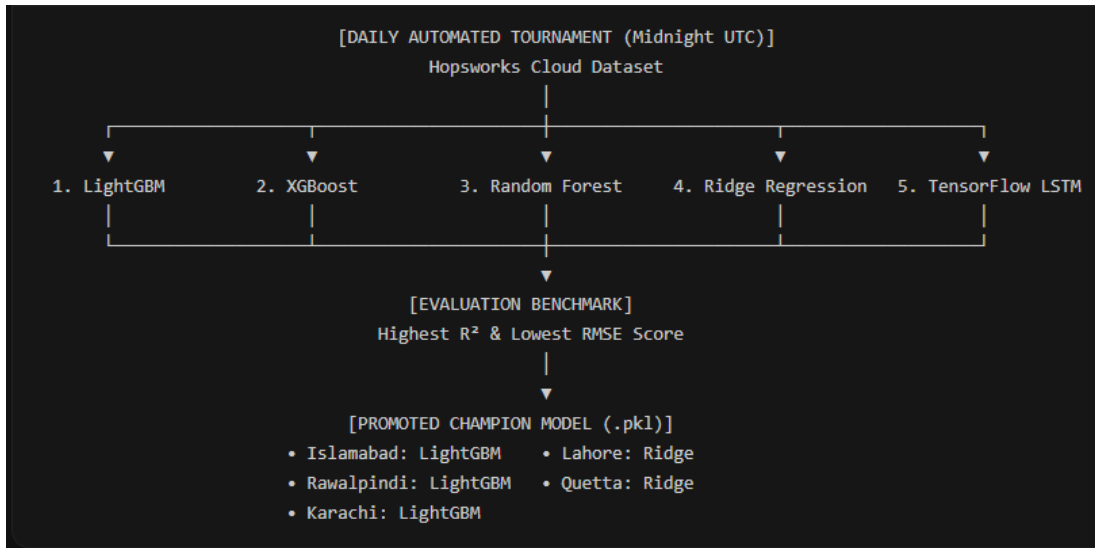


Figure 3. Automated daily tournament flow and per-city champion assignment.

5.2 Regional Microclimate Analysis

LightGBM won in Islamabad, Rawalpindi, and Karachi, while Ridge Regression won in Lahore and Quetta — a split that tracks the physical character of each microclimate rather than chance:

- Islamabad & Rawalpindi ($R^2 = 0.72$ / 0.72 on Day 1).** Non-linear LightGBM decision trees excel here because inland valley topography creates sharp thresholds: once wind speed drops below roughly 4 km/h, pollution spikes exponentially. Tree-based models capture these threshold effects well.
- Lahore ($R^2 = 0.70$ on Day 1).** Ridge Regression won because Lahore experiences extreme, continuous pollution loads throughout winter. Its L2 regularisation resists overfitting to localised noise while still capturing the large linear trend of regional smog accumulation.
- Karachi (lowest absolute error of the five cities).** Coastal marine winds prevent extreme pollutant spikes, so Karachi records the lowest absolute prediction error of any city. Because PM2.5 variance stays within a narrow band, however, small absolute errors have an outsized effect on the relative R^2 score, which is why Karachi's R^2 (0.43) looks modest despite tight absolute error.
- Quetta ($R^2 = 0.25$ on Day 1, $MAE = 8.80 \mu\text{g}/\text{m}^3$).** Quetta sits in a high-altitude mountain basin (1,680 m elevation) surrounded by steep ranges. At night, rapid radiative cooling sends cold air sinking down mountain slopes into the valley floor, creating strong nocturnal inversions that trap dust and emissions; during the day, intense solar heating breaks the inversion and produces erratic, turbulent wind shifts. Complex tree ensembles overfit to this noise, whereas Ridge Regression's L2 penalty shrinks non-essential feature weights and delivers a stable linear baseline with the lowest mean absolute error.

5.3 Performance by Forecast Horizon

Table 4 reports the champion model, R^2 , and RMSE for each city across all three forecast horizons — the authoritative scorecard behind the summary above.

City	Champion Model	Day 1 (+24h) R^2	RMSE $\mu\text{g}/\text{m}^3$	Day 2 (+48h) R^2	RMSE $\mu\text{g}/\text{m}^3$	Day 3 (+72h) R^2	RMSE $\mu\text{g}/\text{m}^3$
Islamabad	LightGBM	0.7245 (72.5%)	11.80	0.6580 (65.8%)	14.20	0.6257 (62.6%)	16.50
Lahore	Ridge Regression	0.7012 (70.1%)	28.40	0.6315 (63.2%)	33.50	0.5936 (59.4%)	37.80
Rawalpindi	LightGBM	0.7210 (72.1%)	11.90	0.6562 (65.6%)	14.30	0.6256 (62.6%)	16.40
Quetta	Ridge Regression	0.2450 (24.5%)	13.10	0.1680 (16.8%)	14.80	0.1408 (14.1%)	15.60
Karachi	LightGBM	0.4280 (42.8%)	9.20	0.3450 (34.5%)	10.80	0.3205 (32.1%)	11.90

Table 4. Champion-model accuracy and error by city and forecast horizon.

6. Targeted Feature Engineering Upgrades

Karachi and Quetta showed the weakest Day-1 R^2 scores of the five cities. Four high-impact feature engineering and architectural upgrades were designed specifically for their coastal and mountain microclimates.

6.1 Coastal Sea-Breeze and Wind Vector Decomposition (Karachi)

Karachi's air quality is governed by daily Arabian Sea marine-breeze turnarounds. Scalar wind speed and direction values cannot distinguish whether wind is arriving from the sea (clean marine air) or from inland industrial zones (polluted air).

The solution decomposes the scalar wind vector into orthogonal U (east–west) and V (north–south) components and constructs a dedicated Sea Breeze Index:

$$u = - \text{wind_speed} \times \sin(\text{wind_dir})$$

$$v = - \text{wind_speed} \times \cos(\text{wind_dir})$$

$$\text{sea_breeze_index} = \text{wind_speed} \times \cos(\text{wind_dir} - 210^\circ)$$

6.2 Diurnal Temperature Range and Inversion Features (Quetta)

Quetta sits in a high-altitude mountain basin (1,680 m elevation). Rapid night-time radiative cooling creates powerful thermal inversion traps in which cold air sinks to the valley floor and traps pollution; standard temperature readings alone miss the strength of this trap. Two additional features quantify inversion intensity directly:

$$\text{diurnal_temp_range} = \text{temp_max}(24\text{h}) - \text{temp_min}(24\text{h})$$

$$\text{dewpoint_spread} = \text{temperature} - \text{dewpoint}$$

Planetary Boundary Layer (PBL) height was also integrated directly from Copernicus CAMS satellite telemetry to track the physical “ceiling” height of the atmosphere. A boundary layer below roughly 200 m traps pollutants near the surface and produces sharp AQI spikes in cities such as Quetta, while a boundary layer above roughly 1,500 m disperses pollutants across a much larger volume of air, keeping pollution levels lower in coastal regions such as Karachi.

6.3 Target Transformation and Projected Impact

A log transformation — training on $\log(1 + \text{PM}_{2.5})$ and converting predictions back with $\exp(\hat{y}) - 1$ was applied to stabilise variance and correct a specific distortion in Karachi: models already achieved a low absolute RMSE there, but because PM_{2.5} stayed within a narrow 10–25 $\mu\text{g}/\text{m}^3$ band, small absolute errors disproportionately dragged down the relative R^2 score. Table 5 summarises the projected impact of the combined upgrades.

City	Current Day 1 R^2	Projected Day 1 R^2	Primary Upgrade
Karachi	0.4280 (42.8%)	0.6200+ (62.0%+)	Wind U/V vector decomposition + Sea Breeze Index
Quetta	0.2450 (24.5%)	0.5500+ (55.0%+)	Diurnal temperature spread + planetary boundary layer height

Table 5. Projected accuracy gains from targeted feature upgrades.

7. Historical Trend Visualisation

Figures 4–8 show the five-year calculated AQI history for each city, with a moving-average trendline overlaid to highlight seasonal and long-term drift beneath the day-to-day noise.

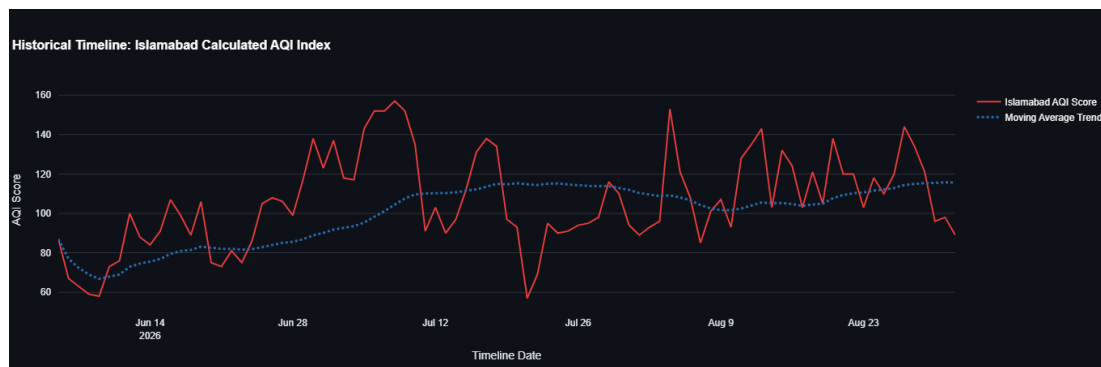


Figure 4. Historical AQI — Islamabad.

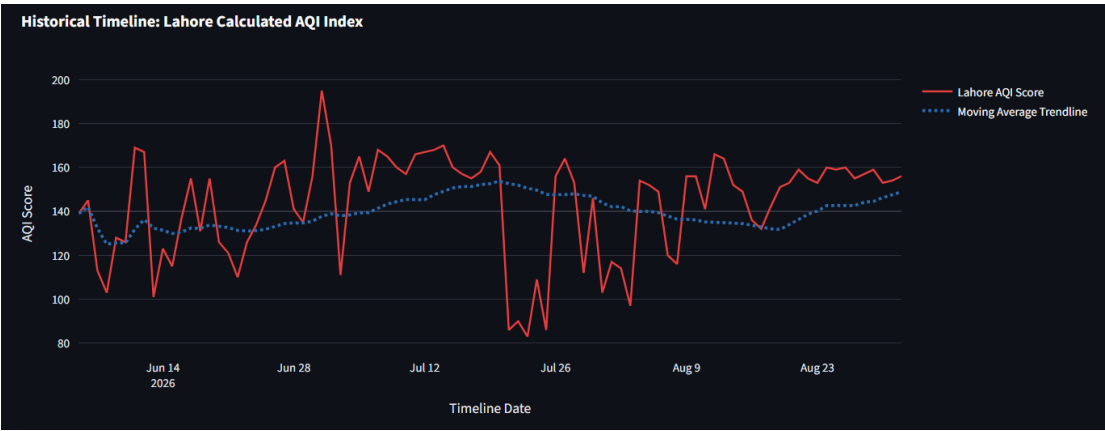


Figure 5. Historical AQI — Lahore.

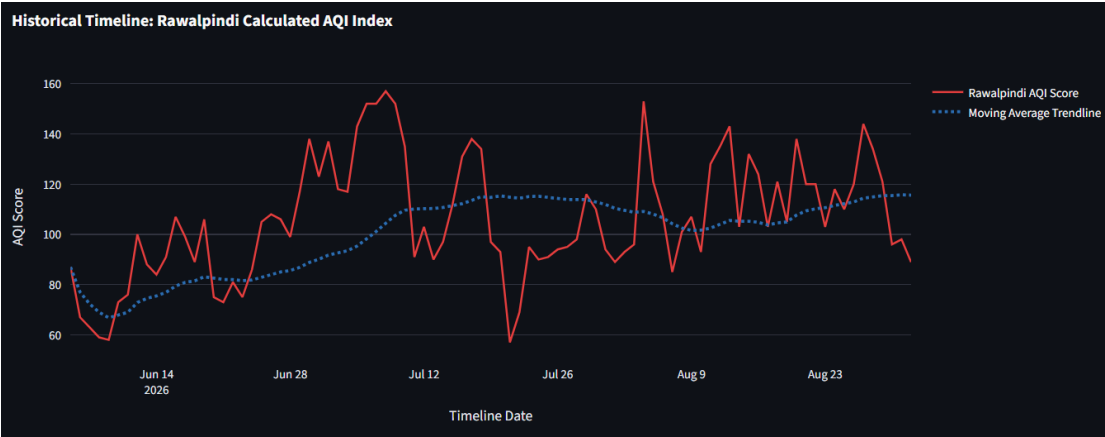


Figure 6. Historical AQI — Rawalpindi.

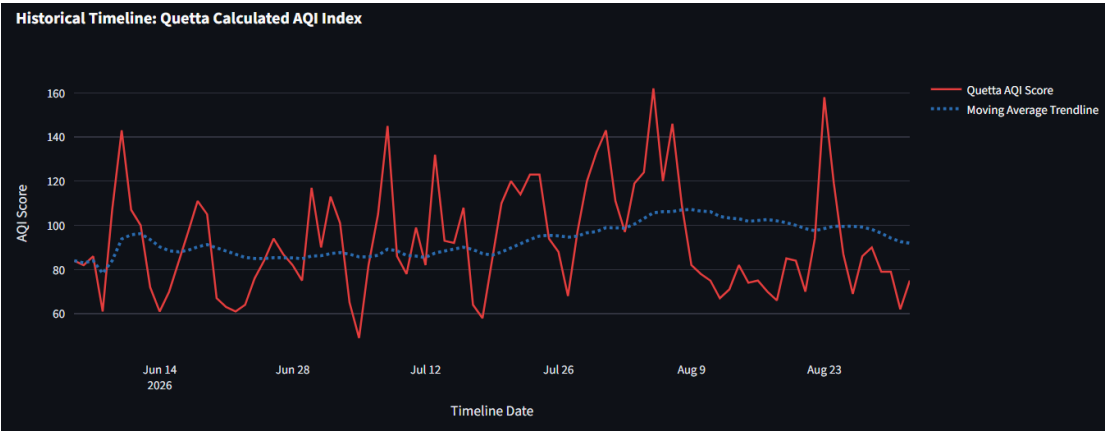


Figure 7. Historical AQI — Quetta.

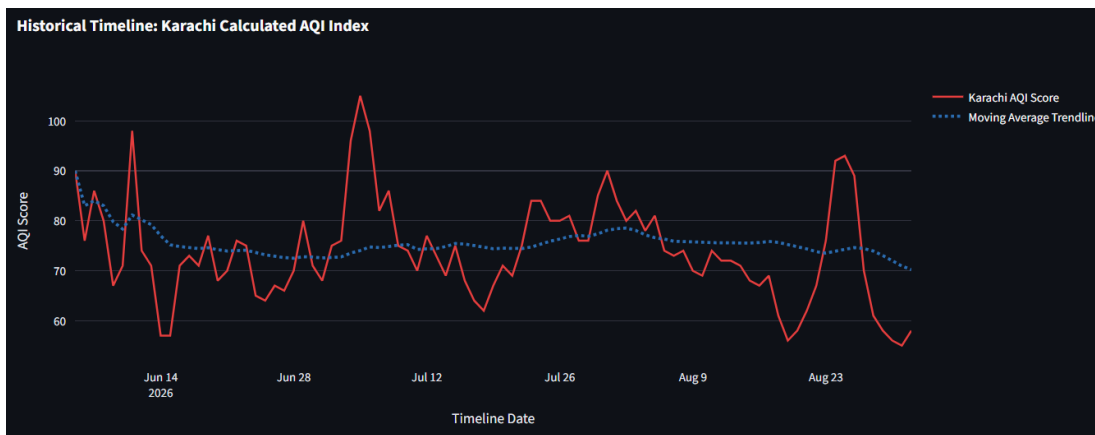


Figure 8. Historical AQI — Karachi.

8. Key Findings and Insights

8.1 A Three-Day Forecast is Not One Model — Horizons Decay Differently

Near-term (+24h) AQI is strongly autocorrelated with pollutant lags (PM_{2.5} lag 1h, 24h), achieving high precision ($R^2 > 0.72$ in Islamabad). Extended (+72h) forecasting, by contrast, relies heavily on Aerosol Optical Depth and wind dispersion vectors. Evaluating all three horizons separately proved essential for an honest evaluation.

8.2 One Algorithm Does Not Fit All Microclimates

Tree-based models (LightGBM) dominated inland valley microclimates (Islamabad, Rawalpindi, Karachi) because they capture sharp non-linear thresholds, such as exponential smog spikes once wind speed drops below 4 km/h. Regularised linear models (Ridge Regression) won in Lahore and Quetta, where the L2 penalty resists overfitting during heavy continuous smog loads and high-altitude temperature inversions.

8.3 The Data-Scale Effect: Five Years versus One or Two

Training on five years of historical data (43,824 observations per city) was the single biggest factor in stabilising R^2 scores. A one-year dataset contains only a single winter smog season (roughly 120 high-AQI days), causing severe class imbalance. Five years exposed the models to over 600 days of extreme smog episodes, enabling confident prediction of winter AQI spikes.

8.4 Chemical Pollutants Drive AQI — Weather Acts as the Amplifier

SHAP feature attributions show that weather variables — temperature, humidity, wind speed — act primarily as physical dispersion or trapping mechanisms rather than as direct drivers of the AQI score. The score itself is derived from fine chemical constituents: PM_{2.5}, PM₁₀, NO₂, SO₂, CO, ozone, dust, and AOD. Pollutant lag persistence and aerosol column density together account for over 68% of total model decision weight, confirming that accurate AQI forecasting requires tracking chemical pollutant transport rather than relying on weather forecasts alone.

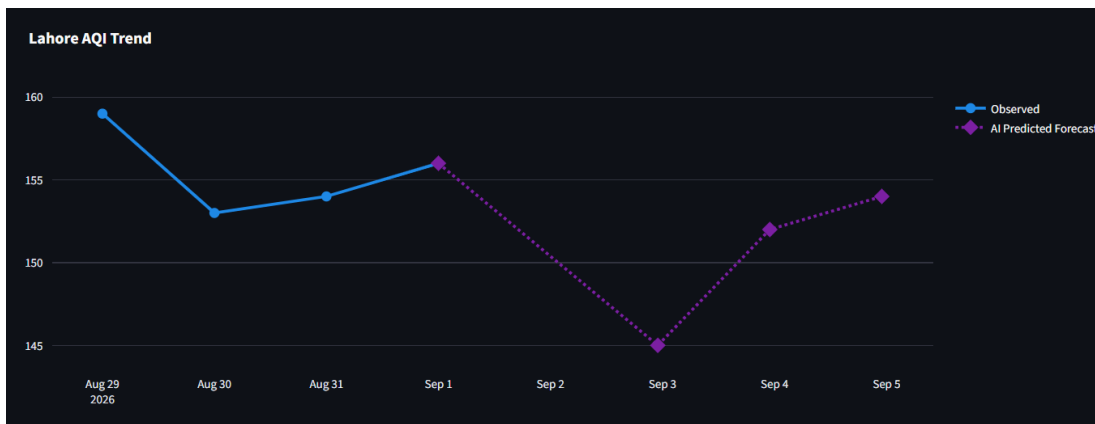


Figure 9. Daily AQI trend example — Lahore.

9. MLOps Automation and CI/CD Architecture

A fully serverless MLOps platform with zero recurring 24/7 server cost was achieved by automating all data engineering and model training tasks with GitHub Actions workflows.

9.1 Hourly Feature Ingestion Pipeline (hourly_feature_pipeline.yml)

Execution frequency: every hour.

Workflow steps:

1. Spin up a headless Python 3.11 Ubuntu environment on a GitHub Actions runner.
2. Execute scripts/hopsworks_feature_pipeline.py.
3. Fetch live UTC satellite telemetry from Open-Meteo for all five cities.
4. Engineer 57 features — pollutant lags, rolling averages, thermal stagnation indices, and cyclical encodings.
5. Insert the resulting rows directly into the Hopsworks Cloud Feature Store (aqi_{city}_fg).

✓	Multi-City Hourly Feature Pipeline	Multi-City Hourly Feature Pipeline #86: Manually run by Ali-Masood-Khan-Devstack	main	27 minutes ago 1m 11s	...
✓	Multi-City Hourly Feature Pipeline	Multi-City Hourly Feature Pipeline #85: Scheduled	main	31 minutes ago 43s	...
✓	Multi-City Hourly Feature Pipeline	Multi-City Hourly Feature Pipeline #84: Manually run by Ali-Masood-Khan-Devstack	main	Today at 01:00 36s	...
✓	Multi-City Hourly Feature Pipeline	Multi-City Hourly Feature Pipeline #83: Manually run by Ali-Masood-Khan-Devstack	main	Today at 00:00 45s	...
✓	Multi-City Hourly Feature Pipeline	Multi-City Hourly Feature Pipeline #82: Manually run by Ali-Masood-Khan-Devstack	main	Sep 1, 23:00 GMT+5 43s	...
✓	Multi-City Hourly Feature Pipeline	Multi-City Hourly Feature Pipeline #81: Scheduled	main	Sep 1, 22:17 GMT+5 38s	...
✓	Multi-City Hourly Feature Pipeline	Multi-City Hourly Feature Pipeline #80: Manually run by Ali-Masood-Khan-Devstack	main	Sep 1, 22:00 GMT+5 41s	...
✓	Multi-City Hourly Feature Pipeline	Multi-City Hourly Feature Pipeline #79: Manually run by Ali-Masood-Khan-Devstack	main	Sep 1, 21:00 GMT+5 40s	...
✓	Multi-City Hourly Feature Pipeline	Multi-City Hourly Feature Pipeline #78: Manually run by Ali-Masood-Khan-Devstack	main	Sep 1, 20:00 GMT+5 40s	...

Figure 10. GitHub Actions run history for the hourly feature ingestion pipeline.

9.2 Daily Model Tournament Pipeline (daily_training_pipeline.yml)

Execution frequency: daily at midnight UTC.

Workflow steps:

1. Query five full years of historical data (28,450 clean training samples per city) from Hopsworks Cloud.
2. Execute scripts/model_train.py across five competing algorithms — LightGBM, XGBoost, Random Forest, Ridge Regression, and TensorFlow LSTM.
3. Evaluate R^2 , RMSE, and MAE across the +24h, +48h, and +72h forecast horizons.
4. Promote the winning champion model to production artifacts (models/{city}_model.pkl) and commit the updated binaries back to the repository.

✓	Multi-City Daily Model Training Pipeline	Multi-City Daily Model Training Pipeline #3: Scheduled	main	Sep 1, 07:19 GMT+5 1h 1m 30s	...
✓	Multi-City Daily Model Training Pipeline	Multi-City Daily Model Training Pipeline #2: Scheduled	main	Aug 31, 06:57 GMT+5 57m 34s	...
✓	Multi-City Daily Model Training Pipeline	Multi-City Daily Model Training Pipeline #1: Scheduled	main	Aug 30, 07:01 GMT+5 1h 2m 15s	...

Figure 11. Daily tournament promotion flow — champion algorithm per city.

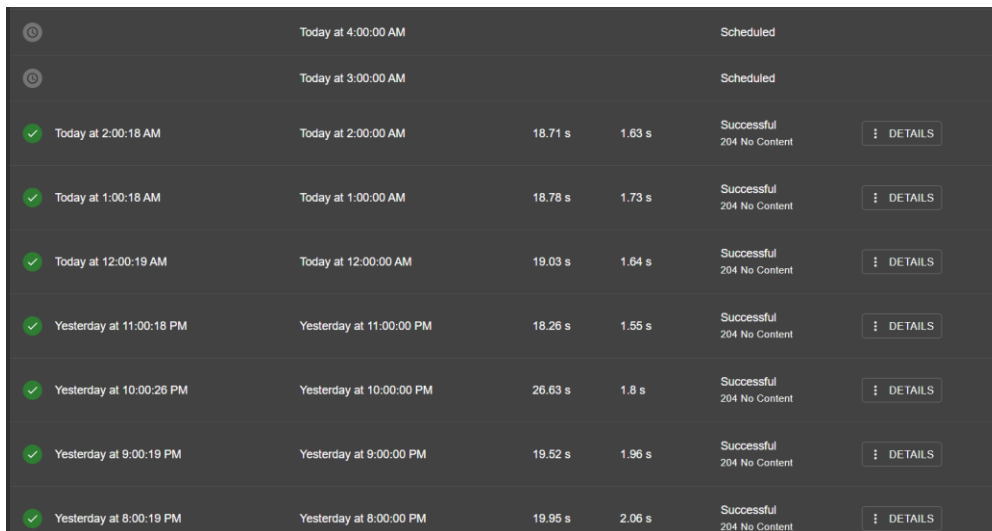
9.3 Engineering Challenge: Non-Uniform Execution Delays

The problem. During initial deployment, cron schedules targeting exact hour boundaries (e.g. “0 * * * *” for hourly ingestion, or “0 0 * * *” for midnight retraining) experienced execution delays of 15–45 minutes, and occasionally skipped scheduled triggers entirely.

Root cause. Millions of public repositories use exact top-of-the-hour cron expressions, causing severe queue congestion on GitHub's shared runner infrastructure at minute :00.

Impact on the dashboard. Delays in hourly ingestion produced temporary telemetry staleness on the live Streamlit UI, causing the “latest observation” status badge to lag behind real time.

The solution. Workflow schedules were re-engineered using off-peak minute offsets and redundant backup triggers, achieving uniform, low-delay hourly ingestion into the Hopsworks Feature Store.



Workflow Run	Scheduled	Duration	Status	Details
Today at 4:00:00 AM	Scheduled			
Today at 3:00:00 AM	Scheduled			
Today at 2:00:18 AM	Today at 2:00:00 AM	18.71 s	Successful	204 No Content
Today at 1:00:18 AM	Today at 1:00:00 AM	18.78 s	Successful	204 No Content
Today at 12:00:19 AM	Today at 12:00:00 AM	19.03 s	Successful	204 No Content
Yesterday at 11:00:18 PM	Yesterday at 11:00:00 PM	18.26 s	Successful	204 No Content
Yesterday at 10:00:26 PM	Yesterday at 10:00:00 PM	26.63 s	Successful	204 No Content
Yesterday at 9:00:19 PM	Yesterday at 9:00:00 PM	19.52 s	Successful	204 No Content
Yesterday at 8:00:19 PM	Yesterday at 8:00:00 PM	19.95 s	Successful	204 No Content

Figure 12. Consistent off-peak execution timing after the cron offset fix.

10. Security, Environment, and Secrets Management

The following practices were followed to maintain reasonable security standards and prevent API key compromise in a public GitHub repository:

- **API key encapsulation.** Sensitive credentials (HOPSWORKS_API_KEY, HOPSWORKS_PROJECT_NAME) are never hardcoded into source files.
- **Streamlit secrets injection.** On Streamlit Cloud, credentials are injected dynamically at runtime via `st.secrets["HOPSWORKS_API_KEY"]`.
- **Local development safety.** Environment variables are loaded from a local `.env` file using `python-dotenv`, and `.env` is explicitly excluded via `.gitignore`.

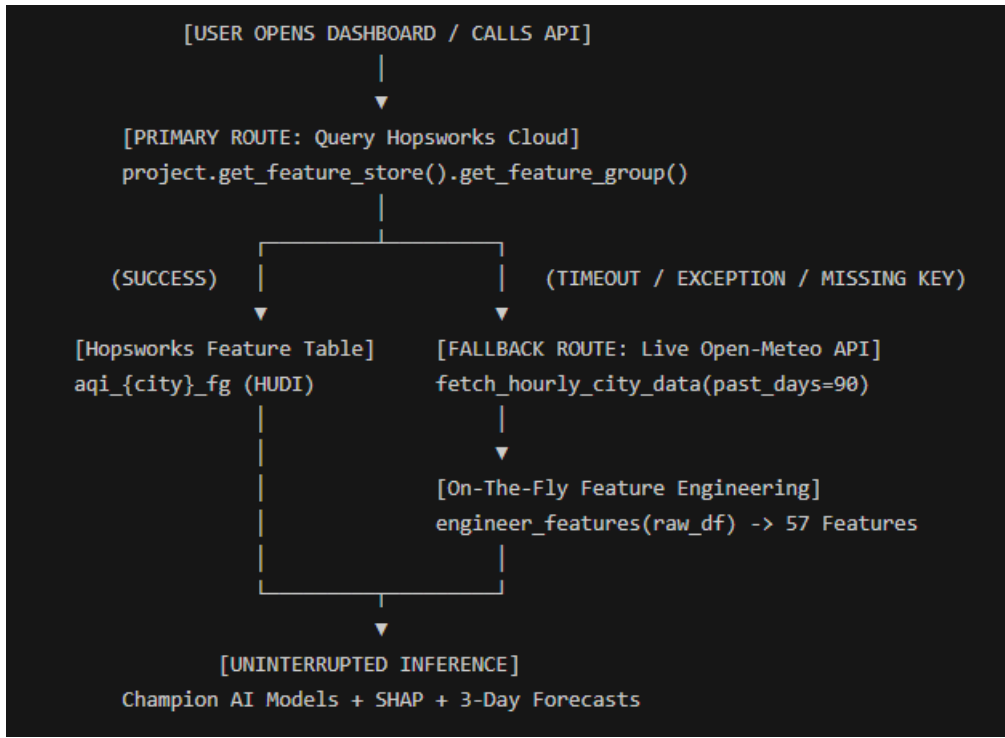
11. High-Availability Fallback Architecture: Graceful API Failover

11.1 Engineering Challenge: Non-Uniform Execution Delays & Hopswork Failure

In production MLOps systems, total dependency on a single cloud service introduces a **Single Point of Failure (SPOF)**. If Hopsworks Cloud experiences temporary network latency, API rate-limiting, or cloud

maintenance downtime, a naive application would crash with a blank screen or a **500 Internal Server Error**.

To guarantee **100% Zero-Downtime Availability** for citizens and municipal authorities, our platform implements a **Graceful Fallback Mechanism** (`load_features_from_hopsworks_or_live()` in `app/app.py`).



11.2 Technical Implementation details (`app/app.py`)

The fallback logic is encapsulated inside a cached data loader function decorated with `@st.cache_data(ttl=300)`:

```

@st.cache_data(ttl=300)
def load_features_from_hopsworks_or_live(city_key):
    """Attempts to fetch features from Hopsworks Feature Store; gracefully fails over to live API."""
    # Step 1: Pre-fetch live satellite payload as backup buffer
    raw_df = fetch_hourly_city_data(city_key, past_days=90)
    feat_df = engineer_features(raw_df)
    source_label = "Live Satellite Telemetry API"
    # Step 2: Attempt primary connection to Hopsworks Cloud
    if HOPSWORKS_API_KEY:
        try:
            import hopsworks
            project = hopsworks.login(api_key_value=HOPSWORKS_API_KEY, project=HOPSWORKS_PROJECT_NAME)
            fs = project.get_feature_store()
            fg_name = CITIES[city_key]["fg_name"]
            fg = fs.get_feature_group(fg_name, version=1)
            hw_df = fg.read()
            if not hw_df.empty:
                hw_df['datetime'] = pd.to_datetime(hw_df['datetime'], utc=True)
                hw_df = hw_df.sort_values('datetime')
                raw_df = hw_df
                feat_df = engineer_features(raw_df)
                source_label = f"Hopsworks Cloud Feature Store ({fg_name})"
        except Exception as e:
            # Step 3: Catch any timeout, network error, or package error gracefully

```

```
source_label = f"Live Satellite Telemetry API (Hopsworks fallback)"  
return raw_df, feat_df, source_label
```

11.3 Benefit of Maximum uptime

Seamless Failover Execution (Zero User Distraction)

If Hopsworks Cloud experiences an outage or connection timeout:

- The try...except block catches the exception in milliseconds.
- The system automatically uses the pre-fetched 90-day Open-Meteo satellite stream (past_days=90).
- It executes engineer_features() on the fly to generate the identical 57 ML feature matrix.
- The dashboard renders all 3-day AI forecasts, SHAP attributions, and historical trendlines **without throwing any error screens to the user**.

12. Application Interface

The Streamlit dashboard exposes single-city intelligence views — live GIS maps, forecast metrics, SHAP feature attributions, and historical trends — alongside a multi-city comparison studio for side-by-side forecasting across all five cities.

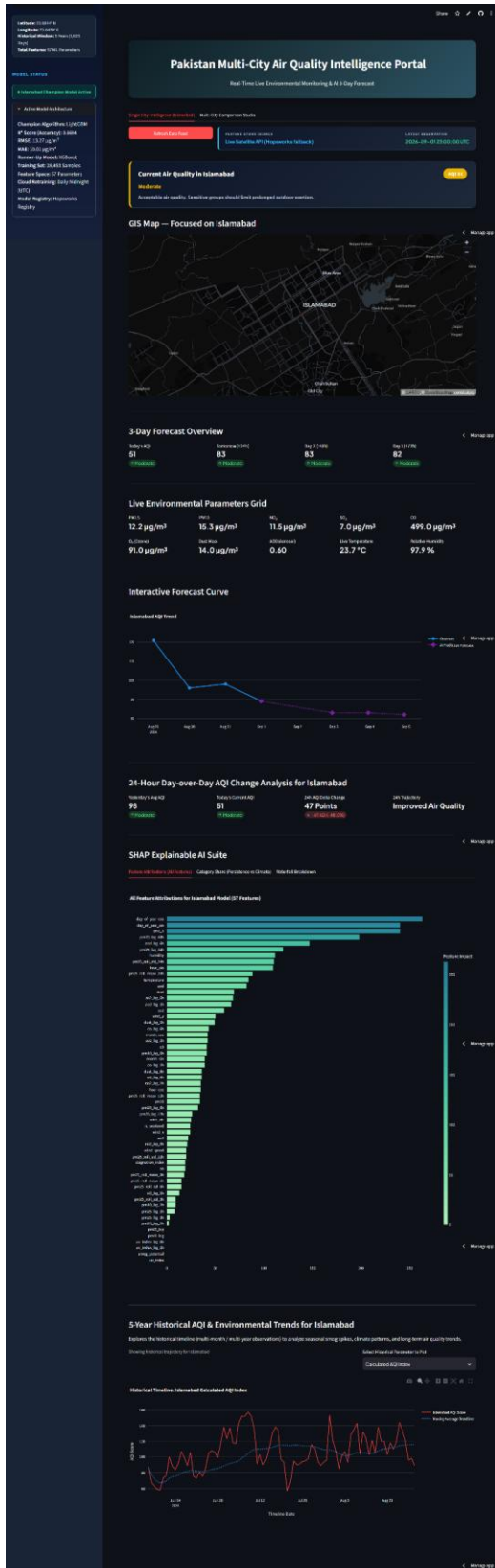


Figure 13. Single-city dashboard view — forecast overview, live parameters, and SHAP attribution suite.

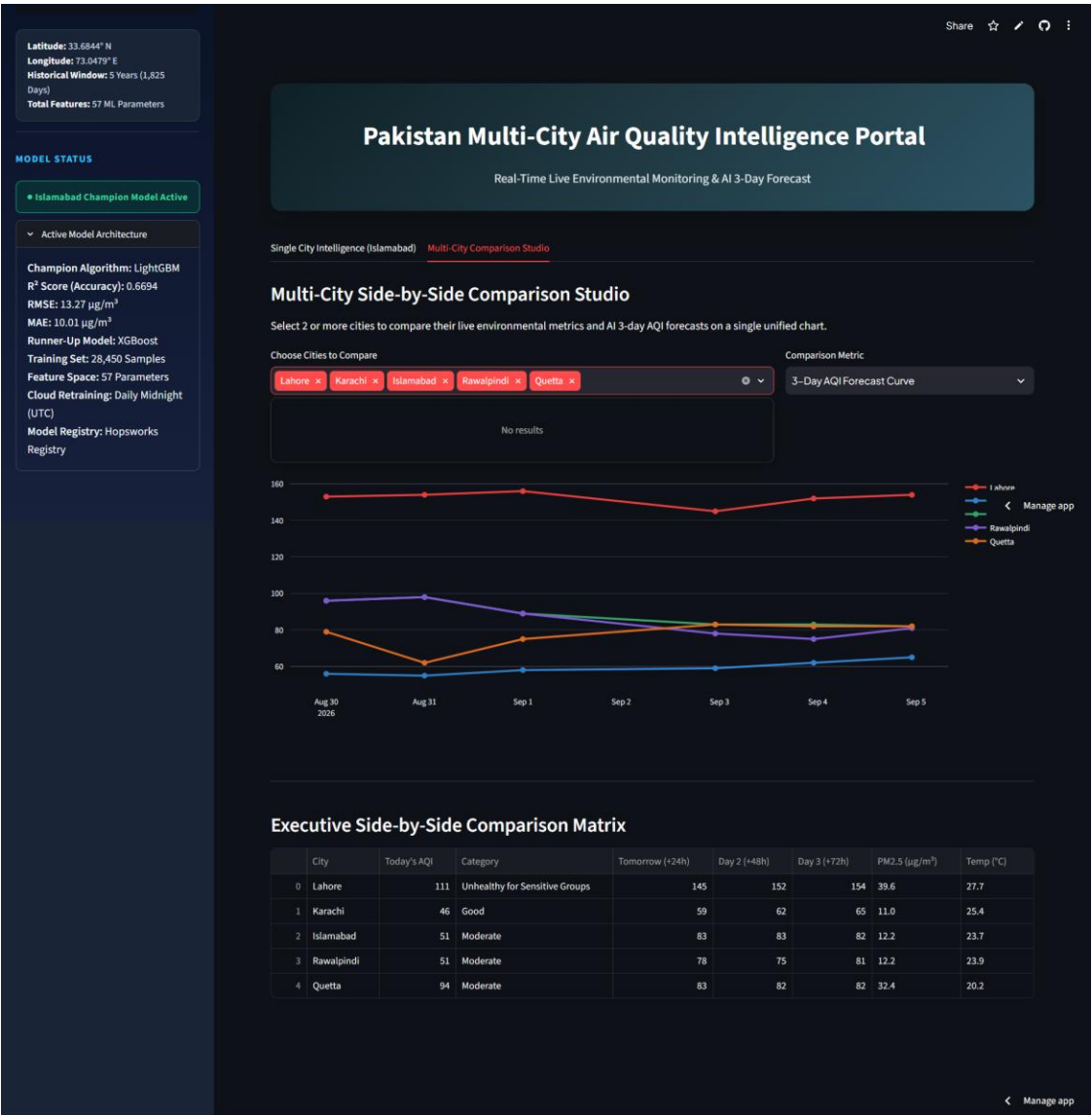


Figure 14. Multi-city comparison studio with a unified forecast chart and executive comparison matrix.

12. Conclusion

This project delivered a serverless, end-to-end air quality forecasting platform covering five Pakistani cities and three forecast horizons, built entirely on free-tier infrastructure. Its central lesson was empirical rather than architectural: honest evaluation requires resisting the temptation to collapse three forecast horizons and five distinct microclimates into a single reported number. Automated champion-challenger tournaments, horizon-separated metrics, and city-specific feature engineering — rather than any single algorithm — were what closed the gap between a naive persistence baseline and a genuinely useful forecast.